

Alphabetic minimax trees

1 Введение

Будем рассматривать двоичные деревья с n листьями. Назовем формой дерева последовательность глубин его листьев в порядке слева направо $\langle l_1, l_2, \dots, l_n \rangle$. Задача поставлена следующим образом: даны веса $w_1, w_2, \dots, w_n$, и по ним нужно построить оптимальное двоичное дерево с n листьями, которое минимизирует

$$\max_{i=1}^n (l_i + w_i)$$

Сам минимум назовем $M(w_1, \dots, w_n)$.

Наш алгоритм будет работать в word RAM model, то есть все элементарные операции с числами из $\log n$ бит исполняются за константное время.

2 Правильные формы

Лемма. Если $\langle l_1, l_2, \dots, l_n \rangle$ - форма дерева и $\max_i l_i > n$, то существует дерево с формой $\langle l'_1, l'_2, \dots, l'_n \rangle$, где $\max_i l'_i \leq n$ и $l'_i \leq l_i \forall i$.

Доказательство. Если есть вершина на глубине $> n$, то на пути от корня до нее есть не менее $n + 1$ вершины. Если степень какой-то из них равна 1, то тогда сжать путь, удалив эту вершину. При этом порядок всех листьев останется прежним. Иначе в каждой из $n + 1$ вершин пути есть ответвление, в котором не менее одного листа. Но листьев всего n . $\square$

Поэтому в оптимальном примере все $l_i \leq n$. В таком случае веса, для которых $w_i \leq \max_i w_i - n$, не влияют на ответ. Сдвинем изначально все веса так, чтобы $\max_i w_i = n$, и теперь будем считать что все $w_i \in [0, n]$.

Последовательность натуральных чисел назовем правильной формой, если она является формой $\langle l_1, l_2, \dots, l_n \rangle$ какого-то дерева. Как по последовательности натуральных чисел понять, является ли она правильной формой или нет?

Давайте введем функцию

$$f_a(x) = \frac{\lceil 2^a x \rceil + 1}{2^a}$$

для целых a .

Теорема (Неравенство Янга). Последовательность натуральных чисел $\langle l_1, \dots, l_n \rangle$ является правильной формой тогда и только тогда, когда

$$f_{l_1} \circ f_{l_2} \circ \dots \circ f_{l_n}(0) \leq 1.$$

Доказательство. Каждой вершине дерева сопоставим отрезок внутри $[0, 1]$ стандартным образом: корень - отрезок $[0, 1]$, для каждой вершины левому ребенку сопоставляется левая половина отрезка, правому ребенку - правая половина отрезка.

Отрезки для листьев не пересекаются по внутренним точкам. Будем расставлять эти отрезки слева направо. Пусть x - координата конца отрезка текущего листа и следующий отрезок - $[y, y + 2^{-l_i}]$. Единственное условие на y - оно должно представляться в виде $\frac{m}{2^i}$, где m целое.

Поэтому

$$y \geq \frac{[2^{l_i}x]}{2^{l_i}},$$

т.к. это наименьшее число, которое не меньше x и представляется в таком виде. Тогда конец нового отрезка оценивается так:

$$y + 2^{-l_i} \geq \frac{[2^{l_i}x] + 1}{2^{l_i}} = f_{l_i}(x),$$

Конец последнего отрезка лежит не правее 1, поэтому нужное неравенство выполнено.

В обратную сторону: выбираем отрезок для каждого листа так, чтобы в неравенствах выше было равенство. Если отрезки для листьев не пересекаются, то найдется двоичное дерево с такими листьями. $\square$

Нам будет удобнее работать с этим неравенством в немного другом виде: введем другую функцию

$$g_a(y) = \left\lceil \frac{y}{2^a} \right\rceil 2^a + 2^a,$$

И проверим, что

$$f_{l_1} \circ f_{l_2} \circ \dots \circ f_{l_n}(0) \leq 1 \quad \Leftrightarrow \quad g_{n-l_1} \circ \dots \circ g_{n-l_n}(0) \leq 2^n.$$

Для этого заметим, что наша новая функция просто масштабирует вычисление f_a в 2^n раз:

$$g_{n-a}(2^n x) = \left\lceil \frac{2^n x}{2^{n-a}} \right\rceil 2^{n-a} + 2^{n-a} = 2^n \frac{[2^a x] + 1}{2^a} = 2^n f_a(x)$$

Для краткости обозначим $g_{a_1, a_2, \dots, a_k} = g_{a_1} \circ g_{a_2} \circ \dots \circ g_{a_k}$.

3 Линейный алгоритм для целых w_i

Мы хотим найти минимальное целое M , для которого существуют целые l_i с условиями

$$l_i + w_i \leq M, \quad g_{n-l_1, \dots, n-l_n}(0) \leq 2^n.$$

Из определения функции g_a видно, что она монотонна по a : $g_{a+1} > g_a$. $l_i \leq M - w_i$, поэтому должно выполняться и неравенство

$$g_{n-M+w_1, \dots, n-M+w_n}(0) \leq 2^n.$$

И наоборот, если такое неравенство выполняется, то положив $l_i = M - w_i$, будет выполняться и условие на правильность формы искомого дерева, т.е. это M достигается.

Итого, мы ищем наименьшее целое M , что

$$g_{n-M+w_1, \dots, n-M+w_n}(0) \leq 2^n.$$

Заметим, что $g_{a+1}(2x) = 2g_a(x)$, тогда при увеличении всех индексов на целое b , значение $g_{n-M+w_1, \dots, n-M+w_n}(0)$ умножится на 2^b . Это значит, что достаточно посчитать $g_{w_1, \dots, w_n}(0)$, а потом найти наименьшее целое C , что

$$g_{w_1, \dots, w_n}(0) \leq 2^C$$

Тогда ответом и будет число C , потому что мы увеличили индексы на $M - n$, а степень двойки стала $2^n \cdot 2^{M-n} = 2^M$.

В итоге, все, что нужно — посчитать композицию $g_{w_1, \dots, w_n}(0)$. Оценим это число из монотонности по индексам:

$$g_{w_1, \dots, w_n}(0) \leq g_{n, \dots, n}(0) = n2^n,$$

Потому что каждый раз g_n прибавляет 2^n к числу $k2^n$. Это число может быть большим. Будем считать композицию представляя числа в таком виде: храним стек номеров битов, на которых стоит 1. (Номер бита не более $n \log n$, поэтому записывается $O(\log n)$ битами). Заметим, что при применении очередного g в таком виде легко посчитать

$$\left\lceil \frac{y}{2^a} \right\rceil 2^a,$$

а также затем прибавить 2^a . Размер стека в итоге увеличивается не более чем на 1, поэтому применение g занимает амортизированную константу.

4 Алгоритмы для вещественных w_i

Лемма.

$$\lfloor M(w_1, \dots, w_n) \rfloor = M(\lfloor w_1 \rfloor, \dots, \lfloor w_n \rfloor).$$

Доказательство.

$$\lfloor M(w_1, \dots, w_n) \rfloor = \min_T \left\lceil \max_i (l_i + w_i) \right\rceil = \min_T \max_i \lfloor l_i + w_i \rfloor = \min_T \max_i (\lfloor w_i \rfloor + l_i) = M(\lfloor w_1 \rfloor, \dots, \lfloor w_n \rfloor).$$

□

Лемма. Пусть $X \in \mathbb{R}$. Тогда

$$M(w_1, \dots, w_n) \leq X \iff M(w'_1, \dots, w'_n) \leq \lfloor X \rfloor,$$

где

$$w'_i = \lfloor w_i \rfloor + \mathbf{1}_{\{w_i\} > \{X\}}.$$

Доказательство. $M(w_1, \dots, w_n) \leq X$ тогда и только тогда, когда для некоторой правильной формы $\langle l_1, \dots, l_n \rangle$ для всех i выполняется $w_i + l_i \leq X$. Дробная часть левой части совпадает с $\{w_i\}$. Поэтому, если $\{w_i\} > \{X\}$, то неравенство равносильно $\lfloor w_i \rfloor + l_i + 1 \leq \lfloor X \rfloor$, а в противном случае неравенство равносильно $\lfloor w_i \rfloor + l_i \leq \lfloor X \rfloor$. Т.е. как раз получается $w'_i + l_i \leq \lfloor X \rfloor$ для всех i . Существование такой правильной формы равносильно $M(w'_1, \dots, w'_n) \leq \lfloor X \rfloor$. □

4.1 Алгоритм за $O(n \log n)$

Таким образом, $\lfloor M(w_1, \dots, w_n) \rfloor$ мы можем найти за линейное время по предыдущему алгоритму. Заметим, что $\{M(w_1, \dots, w_n)\}$ совпадает с $\{w_i\}$ для какого-то i . Поэтому, с помощью последней леммы, можно бинарным перебором дробные части всех w_i в качестве $\{X\}$, каждый раз пользуясь алгоритмом для целых w'_i , таким образом получив алгоритм за $O(n \log n)$.

Все следующие алгоритмы делают то же самое, но в некотором смысле запоминают результат предыдущих вычислений, что позволяет ускорить алгоритм.

4.2 Алгоритм за $O(nd)$, где d - количество различных $\lfloor w_i \rfloor$

Теперь мы хотим считать композицию функций на подотрезках. Для этого нужны следующие две леммы: как такие функции выражаются и как брать их композицию.

Лемма. Для целых $a_1, \dots, a_k$ функция $g_{a_1, \dots, a_k}$ имеет вид

$$g_{a_1, \dots, a_k}(x) = \left\lceil \frac{x+r}{2^a} \right\rceil 2^a + c,$$

где $a = \max_i a_i$, а числа r и c имеют вид

$$\sum_{i=1}^k \alpha_i 2^{a_i},$$

где все $\alpha_i \in \{0, 1, 2\}$.

Лемма. Пусть

$$h_1(x) = \left\lceil \frac{x+r_1}{2^a} \right\rceil 2^a + c_1 \quad \text{и} \quad h_2(x) = \left\lceil \frac{x+r_2}{2^b} \right\rceil 2^b + c_2.$$

Если $b \leq a$, то

$$h_2(h_1(x)) = \left\lceil \frac{x+r_1}{2^a} \right\rceil 2^a + \left\lceil \frac{c_1+r_2}{2^b} \right\rceil 2^b + c_2.$$

Иначе, то есть если $b > a$, то

$$h_2(h_1(x)) = \left\lceil \frac{x+r_1 + \left\lceil \frac{c_1+r_2}{2^a} \right\rceil 2^a}{2^b} \right\rceil 2^b + c_2.$$

Доказательство. Прямая подстановка. Из композиции следует лемма про представление. $\square$

В нашем случае в качестве a_i нас интересуют $w'_i = \lfloor w_i \rfloor$ или $\lfloor w_i \rfloor + 1$.

Пусть $b_1 < b_2 < \dots < b_d$ - все различные значения $\lfloor w_i \rfloor$. Тогда в сумме $\sum_{i=1}^k \alpha_i 2^{a_i}$, где все $\alpha_i \in \{0, 1, 2\}$, показатель степени двойки у каждого слагаемого равен b_j или $b_j + 1$, и таких слагаемых не более n . Поэтому, если отмерить $O(\log n)$ битов влево от бита на позиции b_j для всех j , то такие отрезки гарантированно покроют все ненулевые биты чисел r и c .

Как будем хранить числа a, r, c ? Число a длины $\log n$, а для r, c нам понадобится следующее краткое представление: мы храним d чисел из $O(\log n)$ бит, и указатели на места, где они должны стоять.

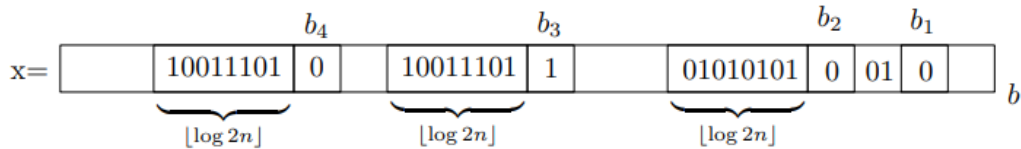


Fig. 1. Succinct representation of x is $[010_b, 010101010_b, 100111011_b, 100111010_b]$.

По кратким представлениям функций h_1, h_2 можно за $O(d)$ посчитать краткое представление их композиции. Потому что операция сложения и операция $\left\lceil \frac{x}{2^a} \right\rceil 2^a$ могут выполняться отдельно для каждого из d чисел представления за константное время.

В начале алгоритма за линейное время мы так же находим целую часть нашего ответа, и затем перебираем бинарным поиском дробную часть X , но при этом на каждом шаге мы будем действовать по-другому. Мы поддерживаем растущие множества L, R , где L — индексы i , для которых мы знаем, что истинное значение w'_i равно $\lfloor w_i \rfloor$, а R — для которых равно $\lfloor w_i \rfloor + 1$, за C обозначим все остальные индексы. Соответственно, когда мы в качестве очередной дробной части в бинарном поиске рассматриваем $\{w_i\}$, то если оказывается, что реальная дробная часть меньше, то все w_j -ые с

такой или большей дробной частью добавляются в R , а в противном случае все w_i с меньшей дробной частью добавляются в L . Когда какой-то индекс i попадает в L или R , для него с этого момента значение w'_i не будет меняться для дальнейших шагов.

Множество $L \cup R$ мы разобьем на максимальные отрезки подряд идущих индексов в нем, и для каждого отрезка мы будем хранить краткое представление композиции соответствующих $g_{w'_i}$. После каждого шага, когда у нас расширяется $L \cup R$, мы склеиваем все рядом стоящие отрезки индексов пока не получим максимальные отрезки подряд идущих индексов. За весь алгоритм на это уйдет суммарно $O(nd)$ времени. Кроме того, на каждом шаге бинарного поиска мы считаем композицию из $O(|C|)$ функций (для максимальных отрезков $L \cup R$ все посчитано, и между двумя максимальными отрезками должен быть индекс из C). Каждый раз $|C|$ падает в 2 раза, поэтому у нас уйдет

$$O\left(nd + \frac{nd}{2} + \frac{nd}{4} + \dots\right) = O(nd)$$

времени, что и требовалось.

4.3 Алгоритм за $O(n)$

Для линейного алгоритма мы поделим индексы от 1 до n на блоки подряд идущих по $\log n$ в каждом. Сузимся на один из блоков. Теперь в сумме $\sum_{i=1}^k \alpha_i 2^{a_i}$ для функции внутри блока не более $\log n$ слагаемых, и каждая степень двойки равна $\lfloor w_i \rfloor$ или $\lfloor w_i \rfloor + 1$. Тогда в выражении чисел r, s композиции функций в пределах одного блока есть только $O(\log n)$ бит, позиции которых можно предподсчитать заранее (теперь они не обязаны идти подряд). Поэтому для каждого блока можно ввести свое краткое представление функций - просто храним число из этих $O(\log n)$, будто они стоят подряд.

Наш алгоритм будет устроен так: сначала внутри каждого блока мы действуем как в алгоритме за $O(nd)$. После этого мы получим $\frac{n}{\log n}$ функций и надо будет отдельно найти их композицию.

Оценим время операций с функциями внутри блоков: На их постепенное склеивание при расширении $L \cup R$ уйдет $O(n)$, потому что теперь мы можем делать композицию за $O(1)$. На очередном шаге бинарного поиска надо посчитать композицию $O(|C| + \frac{n}{\log n})$ функций, потому что разделили наших максимальных отрезков — индексы из C и границы блоков. Тогда на подсчет композиции функций для всех блоков за все шаги уйдет

$$O\left(n + \frac{n}{\log n} + \frac{n}{2} + \frac{n}{\log n} + \frac{n}{4} + \frac{n}{\log n} + \dots\right) = O(n)$$

времени. Осталась основная проблема: на каждом шаге посчитать композицию $\frac{n}{\log n}$ функций для разных блоков за амортизированное $O\left(\frac{n}{\log n}\right)$.

Для разных блоков функции имеют краткие представления на разных битах, поэтому нужно делать по-другому. Для этого вводится гибридное представление числа, которое является упорядоченным списком из объектов вида:

1. Чанк: подряд идущие $\leq \log n$ бит в числе.
2. Разреженный чанк: не более $\log n$ битов, между которыми стоят только нули, причем эти биты в разреженном чанке стоят в точности на позициях, соответствующих подотрезку из позиций краткого представления одного из блоков.

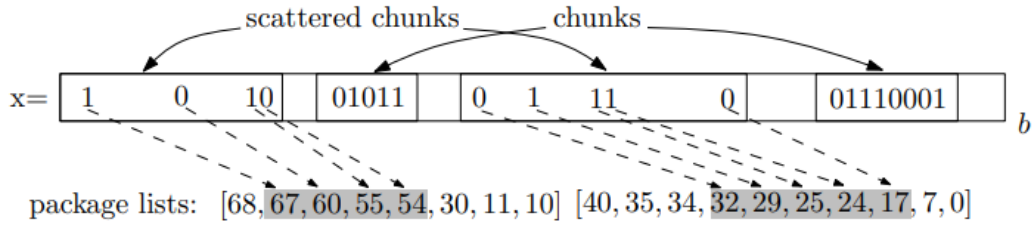


Fig. 2. Example of a valid hybrid representation.

Подробнее: для чанка мы храним его начало, конец и одно число длины $\log n$. Для разреженного чанка мы храним начало, конец, номер блока, концы подотрезка позиций краткого представления для этого блока и число длины $O(\log n)$ в котором записаны все эти биты подряд. Чанки не должны пересекаться и должны покрывать все ненулевые биты числа.

Теперь мы берем $x = 0$ и последовательно применяем к нему функции вида $\lceil \frac{x+r}{2^a} \rceil 2^a + c$. Каждый раз храним гибридное представление числа x . Утверждается, что можно посчитать гибридное представление значения функции за амортизированную константу, что дает нам $O(\frac{n}{\log n})$ времени на шаг бинарного поиска. Поэтому итоговое время работы $O(n)$.